

AWS Hands-On Lab

Understanding Application Load Balancer (ALB) in AWS

Author

Dr. Sandeep Kumar Sharma

PhD | Solution Architect | Cloud & DevOps Trainer

Welcome to the Lab

Hello Team,

In this hands-on exercise, we are going to learn one of the most important services used in modern cloud environments: the **Application Load Balancer (ALB)**.

Rather than just discussing theory, we will build a real-world architecture where multiple web servers serve the same application, and AWS automatically distributes incoming user traffic between them.

By the end of this lab, you will clearly understand how organizations such as Amazon, Netflix, Flipkart, and many others handle thousands or even millions of user requests without overloading a single server.

Learning Objectives

In this lab, you will learn:

- ✓ What is an Application Load Balancer
- ✓ Why organizations use Load Balancers
- ✓ How to deploy multiple web servers
- ✓ How to install and configure Apache Web Server
- ✓ How to create a Target Group
- ✓ How to register EC2 instances into a Target Group

- ✓ How to configure an Application Load Balancer
 - ✓ How AWS performs Health Checks
 - ✓ How traffic is automatically distributed among servers
 - ✓ How High Availability and Fault Tolerance work
-

Learning Outcomes

After completing this lab, you will be able to:

- Deploy a scalable web application architecture
 - Configure an AWS Application Load Balancer
 - Understand request routing at Layer 7 (HTTP/HTTPS)
 - Configure and monitor Target Groups
 - Troubleshoot unhealthy targets
 - Explain ALB architecture in interviews and real projects
-

Real-World Scenario

Let's imagine you have launched an online shopping application.

Initially, only 10 users access the application, and one server is sufficient.

However, during a festive sale, suddenly 10,000 users try to access the website.

If only one server is handling all requests:

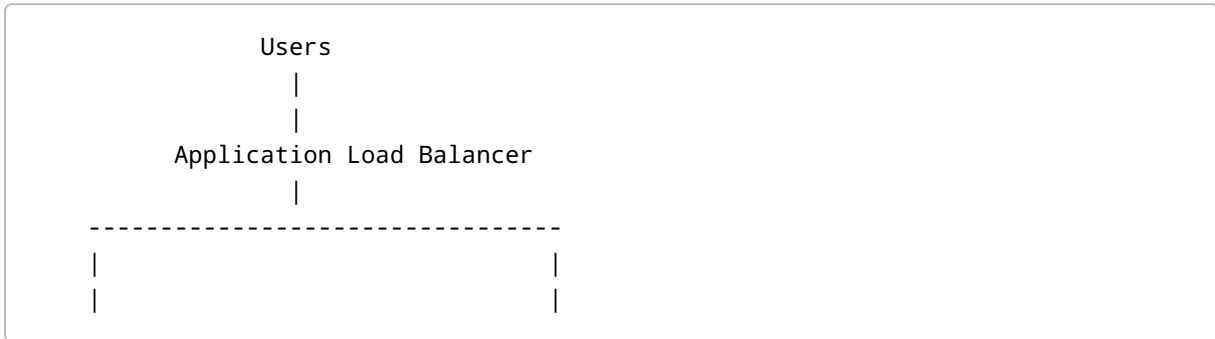
- The server may become slow
- The application may stop responding
- Users may experience downtime

To solve this problem, organizations deploy multiple servers and place a Load Balancer in front of them.

The Load Balancer becomes the single entry point for all users and intelligently distributes traffic among multiple servers.

That is exactly what we are going to build in this lab.

Architecture Overview



Ubuntu Server 1 Ubuntu Server 2 Apache Installed Apache Installed | | | Port 80 Port 80

Lab Environment

We will create:

Component	Quantity
Ubuntu 24.04 EC2 Instance	2
Application Load Balancer	1
Target Group	1
Security Group	1
Apache Web Server	2

Part 1: Launch Web Server 1

Navigate to:

AWS Console → EC2 → Launch Instance

Provide the following values:

Name

WebServer-1

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS

Instance Type

t2.micro

Key Pair

Select your existing key pair

Network Settings

Select your VPC

Select Public Subnet-A

Enable Auto Assign Public IP

Security Group

Create a new Security Group

Name:

Web-SG

Add the following inbound rules:

Type	Port
SSH	22
HTTP	80

Click **Launch Instance**

Wait until the instance state becomes:

Running

Why Did We Open Port 80?

Apache Web Server listens on Port 80.

If Port 80 is blocked:

Users cannot access the website.

Therefore, we must allow incoming HTTP traffic.

Part 2: Launch Web Server 2

Repeat the same process.

Use:

Name:

WebServer-2

Place it in a different subnet if available.

Launch the instance.

Wait until:

Running

Part 3: Install Apache on Server 1

Connect using SSH:

```
ssh -i mykey.pem ubuntu@Public-IP
```

Update Ubuntu packages:

```
sudo apt update
```

Install Apache:

```
sudo apt install apache2 -y
```

Start Apache:

```
sudo systemctl start apache2
```

Enable Apache:

```
sudo systemctl enable apache2
```

Verify:

```
sudo systemctl status apache2
```

Expected Result:

Apache service should show:

active (running)

Part 4: Create Custom Web Page on Server 1

Open:

```
sudo nano /var/www/html/index.html
```

Replace everything with:

```
<html>
<body>
<h1>Welcome to Web Server 1</h1>
<h2>AWS Application Load Balancer Demo</h2>
</body>
</html>
```

Save and Exit.

Open browser:

```
http://Public-IP-Server1
```

Expected Output:

Welcome to Web Server 1

Why Are We Creating Different Pages?

This is very important.

If both servers show exactly the same content, we cannot determine which server is responding.

Therefore:

Server 1 displays:

Welcome to Web Server 1

Server 2 displays:

Welcome to Web Server 2

This helps us visually verify traffic distribution.

Part 5: Install Apache on Server 2

Connect to Server 2.

Run:

```
sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
```

Part 6: Create Custom Page on Server 2

Edit:

```
sudo nano /var/www/html/index.html
```

Paste:

```
<html>
<body>
<h1>Welcome to Web Server 2</h1>
<h2>AWS Application Load Balancer Demo</h2>
</body>
</html>
```

Verify using browser.

Expected Output:

Welcome to Web Server 2

Part 7: Create a Target Group

Navigate to:

EC2 → Target Groups

Click:

Create Target Group

Configuration

Target Type:

Instance

Target Group Name:

Web-TG

Protocol:

HTTP

Port:

80

VPC:

Select your VPC

Click Next

What is a Target Group?

Think of a Target Group as a container.

Instead of connecting directly to servers, the Load Balancer sends traffic to a Target Group.

The Target Group then manages all registered servers.

Flow:

User → ALB → Target Group → EC2 Instance

Register Servers

Select:

☒ WebServer-1

☒ WebServer-2

Click:

Include as Pending Below

Click:

Create Target Group

Part 8: Verify Health Checks

Open:

Target Groups → Web-TG

Select:

Targets

Initially you may see:

Initial

After 1–2 minutes:

Healthy

What Is a Health Check?

AWS periodically sends requests to:

```
http://server-ip:80
```

If the server responds correctly:

Healthy

If the server does not respond:

Unhealthy

Only healthy servers receive traffic.

Part 9: Create Application Load Balancer

Navigate to:

EC2 → Load Balancers

Click:

Create Load Balancer

Choose:

Application Load Balancer

Basic Configuration

Name

Web-ALB

Scheme

Internet Facing

IP Address Type

IPv4

What Does Internet Facing Mean?

Internet Facing means:

Users from the internet can access the Load Balancer.

Without this option:

The ALB would only be accessible internally.

Network Mapping

Select your VPC.

Select at least two Availability Zones.

Example:

☒ Subnet-A

☒ Subnet-B

Why Two Availability Zones?

Suppose one Availability Zone becomes unavailable.

AWS automatically serves traffic from the second Availability Zone.

This improves availability and fault tolerance.

Security Group for ALB

Create or Select:

ALB-SG

Allow:

HTTP → Port 80 → Anywhere

Listener Configuration

Protocol:

HTTP

Port:

80

Default Action:

Forward To

Web-TG

Click:

Create Load Balancer

Wait until status becomes:

Active

Part 10: Test the Load Balancer

Open:

Load Balancer

Copy:

DNS Name

Example:

```
http://web-alb-xxxx.ap-south-1.elb.amazonaws.com
```

Paste into browser.

Refresh multiple times.

Expected Results:

Refresh 1:

Welcome to Web Server 1

Refresh 2:

Welcome to Web Server 2

Refresh 3:

Welcome to Web Server 1

Refresh 4:

Welcome to Web Server 2

Congratulations!

Traffic is now being distributed by AWS.

What Happened Behind the Scenes?

Whenever a user accesses the ALB URL:

Step 1: Request reaches ALB

Step 2: ALB checks healthy servers

Step 3: ALB selects one target

Step 4: Request is forwarded

Step 5: Server responds

Step 6: Response is returned to the user

All of this happens in milliseconds.

Part 11: Demonstrate Fault Tolerance

Let's simulate a server failure.

Connect to Server 1.

Stop Apache:

```
sudo systemctl stop apache2
```

Wait approximately 2 minutes.

Open:

Target Group → Targets

Observe:

Server 1 = Unhealthy

Server 2 = Healthy

Refresh ALB URL.

You should now see:

Welcome to Web Server 2

every time.

What Did We Learn?

Even though one server failed:

- ✓ Website remained available
- ✓ Users experienced no downtime
- ✓ AWS automatically removed unhealthy server
- ✓ Traffic continued to healthy server

This is called Fault Tolerance.

Interview Questions

Q1. What is an Application Load Balancer?

Q2. Why do we need a Load Balancer?

Q3. What is a Target Group?

Q4. What is a Health Check?

Q5. What happens when a target becomes unhealthy?

Q6. Why should ALB be deployed across multiple Availability Zones?

Q7. What is the difference between ALB and NLB?

Lab Summary

In this lab, we deployed two Ubuntu 24.04 web servers, installed Apache, configured custom web pages, created a Target Group, registered both servers, and deployed an Application Load Balancer. We verified request distribution, health checks, and automatic failover. This architecture forms the foundation of highly available and scalable web applications used in modern cloud environments.